

INGOBOKA

Technical System Blueprint

Enterprise B2B2C Digital Insurance Platform for Rwanda

Document purpose	Concise technical definition of how the platform works
Backend direction	Java 21 + Spring Boot + PostgreSQL + Flyway
Delivery approach	Modular monolith for MVP; service boundaries ready for scale
Status	Editable working blueprint

Working name: “Ingoboka” — protection and support when it matters most.

0. Purpose and Technical Positioning

This document defines the operating model, service boundaries, user responsibilities, best-case business flow, enterprise technology approach, and proposed database structure for Ingoboka. It is intentionally shorter and more technical than the overall concept document.

Architecture decision: Build the hackathon MVP as one Spring Boot application with clearly separated modules. This reduces delivery risk in six days while preserving boundaries that can later be extracted into microservices without redesigning the business model.

1. Problem Description

Many Rwandan citizens—especially informal workers, small traders, moto riders, young adults and households with irregular income—remain exposed to financial shocks because insurance products are difficult to discover, understand, pay for and claim against through one simple digital experience.

- **Access gap:** Potential customers do not have one trusted channel to compare relevant, affordable products from participating insurers.
- **Affordability mismatch:** Conventional payment schedules may not match daily or weekly income patterns.
- **Information gap:** Policy benefits, exclusions, waiting periods and claim requirements are often difficult to understand.
- **Process fragmentation:** Enrollment, payment, policy proof, claim evidence and status tracking may happen through disconnected channels.
- **Trust gap:** Customers need clear consent, traceable decisions, explainable rejection reasons and an appeal path.
- **Provider gap:** Insurers and distribution partners need a lower-cost digital channel for serving new customer segments and monitoring operations.

2. Solution Provided

Ingoboka is a B2B2C digital insurance distribution and service platform. Licensed insurers and approved partners configure products on the business side; citizens discover, enroll, pay, hold policies and submit claims on the consumer side. Ingoboka does not underwrite insurance risk or replace the insurer's legal decision-making.

- **For citizens:** guided needs assessment, product comparison, understandable cover summaries, flexible premium schedules, digital policy wallet, QR verification, claim submission and status tracking.
- **For insurers:** product configuration, enrollment review, policy servicing, claim work queues, decision recording, reconciliation, reporting and audit trails.
- **For partners and agents:** assisted onboarding, customer support, referral tracking and controlled access to assigned customers.
- **For the platform:** multi-tenant administration, contract pricing, commission calculation, security controls, integrations, observability and compliance evidence.

3. Why This Solution Is Chosen as the Best

Reason	Why it matters
Direct problem-solution fit	It solves the complete insurance journey rather than adding a single isolated feature.
High demonstration value	Judges can see an end-to-end working flow: product → application → payment → policy → claim → decision.
Inclusive design	Mobile-first, low-data and bilingual experiences can serve customers with different levels of digital literacy.
Enterprise relevance	The same platform creates value for insurers, agents, cooperatives, employers and customers.
Responsible model	Insurers keep underwriting and claim authority; the platform keeps consent, traceability and clear role separation.
Buildable and scalable	A focused modular MVP is realistic in six days, while the domain boundaries support future enterprise growth.

4. B2B2C Income Model

The principal customer paying Ingoboka is the insurer or institutional distribution partner, while the citizen receives the digital service. Revenue must be transparent, contract-based and subject to applicable insurance and financial-sector requirements.

Revenue stream	How it works	Payer
Enterprise onboarding fee	One-time charge for tenant setup, branding, product configuration, integration and training.	Insurer / partner
SaaS subscription	Monthly or annual platform fee based on plan, users, products and service-level requirements.	Insurer / partner
Per-active-policy fee	Small contracted fee for policies actively serviced through the platform.	Insurer
Distribution commission	Percentage or fixed commission on successfully issued policies where legally and contractually allowed.	Insurer
White-label and API fee	Charge for branded portals, embedded enrollment, API usage or dedicated environments.	Insurer / enterprise partner
Premium analytics package	Aggregated operational dashboards and scheduled reports; no sale of personal data.	Insurer / partner

Commercial guardrail: Customer premiums are insurer funds, not Ingoboka revenue. The platform must not hide charges, sell personal data, or earn from denying claims. Commission and platform fees are recorded separately in the revenue ledger.

5. Service Layer

Service	What it provides
Identity & Access	Registration, login, JWT/refresh tokens, OTP, RBAC, tenant-aware authorization and session security.
Partner Management	Insurer/partner onboarding, organization profiles, contracts, staff memberships and service configuration.
Customer Profile & Consent	Citizen profile, KYC status, preferences, dependants, legal consent versions and revocation.
Product Catalog	Insurance products, plans, benefits, exclusions, eligibility, premium frequencies and publication workflow.
Quotation & Enrollment	Needs assessment, suitability rules, quote generation, application submission and review.
Policy Management	Policy issuance, lifecycle status, renewals, members/beneficiaries, documents and secure QR verification.
Billing & Payment	Premium schedules, payment initiation, provider callbacks, allocation, receipts and reconciliation.
Claims & Appeals	Claim intake, evidence, validation, status workflow, insurer decision, reason recording and appeal.
Document Management	Secure upload, malware checks, metadata, checksums, retention and access-controlled download.
Notification	SMS, email, push and in-app templates for payment, policy, claim and support events.
Revenue & Commission	Partner pricing contracts, transaction calculation, invoices, commission ledger and settlement status.
Reporting & Analytics	Operational dashboards, policy/claim/payment reports and exportable partner summaries.

Service	What it provides
Audit & Compliance	Immutable activity trail, consent evidence, access logs, data-subject requests and compliance reports.
Integration Adapters	Payment, messaging, identity verification and insurer-core connectors behind stable interfaces.

6. Users and Roles — Who Does What?

Role	Responsibilities
Citizen / Policyholder	Registers, gives consent, completes profile, compares products, applies, pays, views policy, submits claims and appeals.
Beneficiary / Dependant	Is attached to a policy and may receive benefits; access is limited and controlled by policy rules.
Agent / Field Officer	Assists assigned citizens with onboarding and applications; cannot approve underwriting or claims.
Customer Support Officer	Resolves enquiries, opens tickets, verifies non-sensitive status and escalates issues.
Insurer Product Manager	Creates products/plans, benefits, exclusions, eligibility rules and publication versions.
Underwriter / Enrollment Officer	Reviews applications, requests information and records insurer approval or rejection.
Claims Officer	Reviews claim evidence, updates workflow and prepares recommendations or decisions within authority.
Claims Supervisor	Approves high-value/exception decisions and manages escalations or appeals.
Finance / Reconciliation Officer	Reviews payment callbacks, failed transactions, settlement, refunds and revenue statements.
Partner Administrator	Manages staff, organization settings, assigned products and operational reports for one tenant.
Compliance / Auditor	Reads consent, decisions, access and audit evidence; cannot change operational records.
Platform Administrator	Manages tenants, global configuration, support access and security operations; actions are fully audited.
Integration Client	Machine identity used by trusted systems for scoped API operations; never uses human permissions.

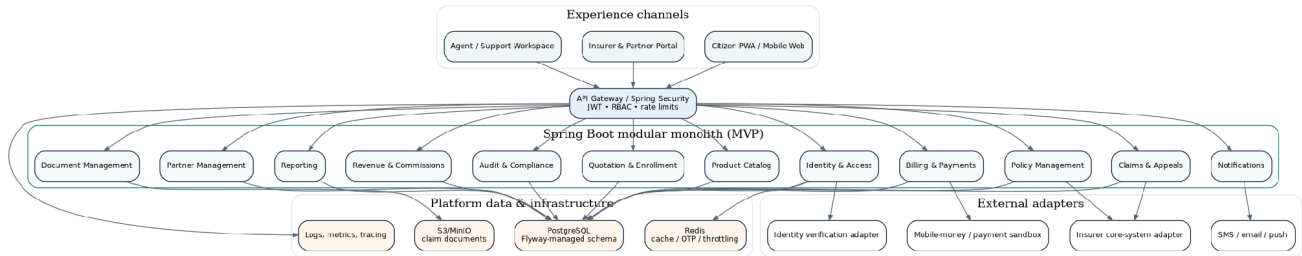
7. Best-Case System Flow

The best-case flow assumes the customer is eligible, the application is approved, payment succeeds and the claim later contains sufficient evidence. Exception paths still exist for rejected applications, failed payments, missing documents and appeals.



Technical event flow: each successful domain action is committed to PostgreSQL together with an outbox event. A background publisher then sends notifications or external integration messages, preventing business data from being lost when an external service is unavailable.

8. Technical Architecture and Technology Use

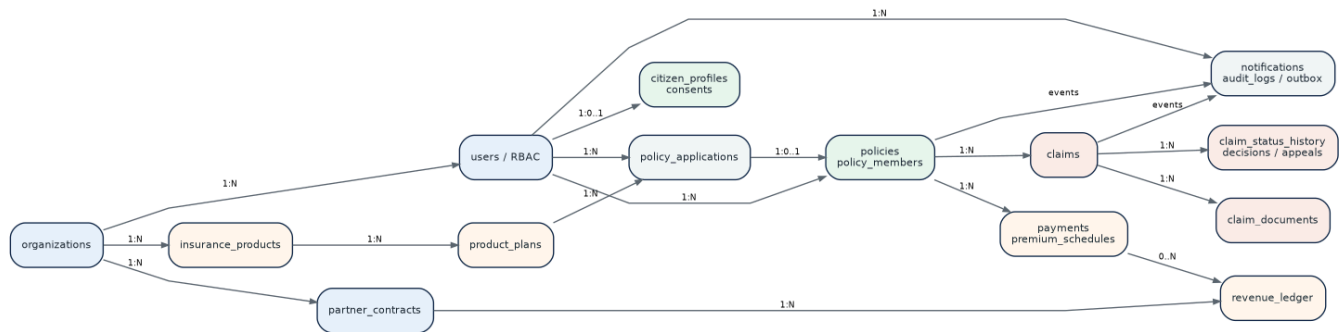


Area	Technology	Use
Frontend	React or Next.js PWA	Citizen mobile-first experience and enterprise web portals; bilingual UI and offline draft support.
Backend	Java 21, Spring Boot	REST APIs, domain modules, validation, transactions, background jobs and integration adapters.
Security	Spring Security, JWT, RBAC	Authentication, tenant-scoped authorization, privileged-role controls and API protection.
Database	PostgreSQL	Transactional source of truth using UUID keys, constraints, indexes and row ownership fields.
Migrations	Flyway	Version-controlled, repeatable and environment-safe database evolution.
Persistence	Spring Data JPA / Hibernate	Repository implementation while domain services own business rules.
Documents	S3-compatible object storage	Claim evidence and policy documents; database stores only metadata and secure object keys.
Cache	Redis	OTP, rate limiting, short-lived sessions and frequently read non-sensitive reference data.
API documentation	OpenAPI / Swagger UI	Developer documentation and testable API contracts.
Quality	JUnit, Testcontainers, Postman/Bruno	Unit, integration, database and end-to-end API validation.
Delivery	Docker, GitHub Actions, managed hosting	Repeatable build, automated tests, environment configuration and deployment.
Observability	Structured logs, metrics, tracing	Production diagnosis, security monitoring and service-level reporting.

Module rule: Modules communicate through application interfaces and domain events—not direct access to another module's repository. This keeps the MVP simple but prevents a tightly coupled monolith.

9. Proposed Final Database Schema

The schema is multi-tenant and relational. All operational tables use UUID primary keys, UTC timestamps, optimistic locking where updates may conflict, explicit foreign keys, check constraints and indexes on status, dates, tenant ownership and business references.



Domain	Tables	Purpose
Identity & tenancy	organizations; users; roles; permissions; user_roles; role_permissions; refresh_tokens	B2B tenant ownership, staff membership and authorization.
Citizen & privacy	citizen_profiles; dependants; consents; data_requests	Customer data, KYC status, consent evidence and privacy workflows.
Products	insurance_products; product_plans; product_benefits; product_exclusions; eligibility_rules	Versioned catalog and pricing definition owned by an insurer.
Enrollment & policy	policy_applications; application_answers; policies; policy_members; policy_documents	From quote/application to issued policy and covered members.
Billing	premium_schedules; payments; payment_events; refunds; reconciliation_batches	Premium obligations, provider callbacks and financial reconciliation.
Claims	claims; claim_documents; claim_status_history; claim_decisions; claim_appeals	Evidence, workflow history, decision, payout value and appeal trail.
B2B2C revenue	partner_contracts; contract_price_rules; revenue_ledger; invoices	Subscription, policy fee, commission and settlement accounting.
Operations	notifications; support_tickets; audit_logs; outbox_events; idempotency_keys	Messaging, support, traceability and reliable integrations.

9.1 Key Database Rules

- Use UUID values generated by the application or PostgreSQL; never expose sequential internal identifiers.
- Every tenant-owned record contains organization_id directly or inherits it through an enforced parent relationship.
- Store national identifiers as encrypted or tokenized values; use a separate deterministic hash only where duplicate detection is required.
- Never store document binaries in PostgreSQL; store object_key, MIME type, size, checksum and access classification.
- Payment callbacks use a unique provider_reference plus idempotency key to prevent duplicate posting.
- Policy number, claim number, product code and organization registration number are unique within their correct scope.
- Status changes are append-only in history tables even when the current status is also stored on the main record.
- Financial records are reversed with compensating entries; they are not silently deleted or overwritten.
- Audit logs are append-only and include actor, tenant, action, entity, timestamp, request correlation ID and safe before/after summaries.

10. Flyway Migration Plan

Migration	Responsibility
V1__enable_extensions_and_base_types.sql	Required extensions, common timestamp conventions and reference types.
V2__create_organizations_and_identity.sql	Organizations, users, roles, permissions, memberships and refresh tokens.
V3__create_customer_and_consent.sql	Citizen profiles, dependants, consents and data requests.
V4__create_product_catalog.sql	Products, plans, benefits, exclusions and eligibility rules.
V5__create_enrollment_and_policy.sql	Applications, answers, policies, members and policy documents.

Migration	Responsibility
V6__create_billing_and_payments.sql	Premium schedules, payments, callbacks, refunds and reconciliation.
V7__create_claims_and_appeals.sql	Claims, evidence metadata, status history, decisions and appeals.
V8__create_partner_revenue.sql	Contracts, pricing rules, revenue ledger and invoices.
V9__create_notifications_support_audit.sql	Notifications, support, audit, outbox and idempotency.
V10__add_constraints_indexes_and_views.sql	Foreign keys, check constraints, unique keys, search indexes and reporting views.
R__refresh_reporting_views.sql	Repeatable creation of database views/functions that may evolve.
Development-only seed	Separate profile-controlled demo data; never run production seed data automatically.

Migration discipline: Never edit a versioned migration after it has been applied to a shared environment. Add a new migration, test upgrade from an empty database and from the previous release, and run Flyway validation during deployment.

11. Core API and Workflow Contracts

Endpoint	Purpose
POST /api/v1/auth/register	Create citizen account and start verification.
POST /api/v1/applications/quote	Generate a quote from plan, profile and assessment inputs.
POST /api/v1/applications	Submit policy application with consent reference.
POST /api/v1/payments	Initiate sandbox/real provider payment through an adapter.
POST /api/v1/payments/webhooks/{provider}	Idempotently process signed provider callbacks.
GET /api/v1/policies/{id}	Return authorized policy summary and lifecycle.
GET /api/v1/policies/verify/{token}	Return minimum public verification data for a valid QR token.
POST /api/v1/claims	Create a claim and return required evidence checklist.
POST /api/v1/claims/{id}/documents	Attach secure document metadata after upload.
PATCH /api/v1/claims/{id}/status	Role-controlled workflow transition with reason.
POST /api/v1/claims/{id}/decision	Record insurer decision and approved amount.
POST /api/v1/claims/{id}/appeals	Open an appeal against an eligible decision.
GET /api/v1/partner/reports/overview	Tenant-scoped operational summary.

Recommended policy states: DRAFT → PENDING_PAYMENT → ACTIVE → GRACE_PERIOD → LAPSED / EXPIRED / CANCELLED. Recommended claim states: DRAFT → SUBMITTED → UNDER_REVIEW → INFORMATION_REQUIRED → APPROVED / REJECTED → PAYMENT_PROCESSING → PAID, with a separate appeal status.

12. MVP Boundary and Acceptance Criteria

The enterprise design is broad, but the hackathon implementation must prove only the highest-value journey. Nonessential modules can be represented by interfaces or seeded data.

Scope	Definition
Must work	Citizen registration; product browsing; quote/application; simulated payment; policy issuance; QR verification; claim submission; insurer review; decision; notification; basic audit.
May be simulated	National-ID verification, mobile-money provider, insurer core system, SMS gateway and claim payout.

Scope	Definition
Should not be built now	Blockchain, facial recognition, complex machine-learning underwriting, full microservices infrastructure or a custom workflow engine.
Definition of done	A deployed system demonstrates one complete customer journey and one insurer journey using separate roles, persistent data, validation, error handling and a reproducible setup.

13. Final Technical Recommendation

Use Ingoboka as a multi-tenant Spring Boot modular monolith backed by PostgreSQL and Flyway. Treat insurer and partner organizations as first-class tenants, keep financial and claim workflows auditable, isolate integrations behind adapters, and build the MVP around one reliable end-to-end journey. This approach is credible for enterprise growth without sacrificing the six-day delivery target.